

目录

子集形式幂级数: inv、ln、exp、pow 与通用复合	2
0. 可直接复制的接口	2
1. 定义与分层变换	2
2. 按集合大小递推求逆	3
2.1 递推式的推导	3
2.2 用分层变换加速递推	4
3. 分层 zeta + Newton 求逆	4
3.1 转化为逐点形式幂级数求逆	4
3.2 Newton 迭代式及精度倍增	7
3.3 只计算新增的高半段	7
3.4 一般 Newton 形式	8
4. 两种实现的区别与复杂度	8
5. ln: 子集卷积对数	8
5.1 定义及变换的正确性	8
5.2 普通幂级数递推的推导	9
6. exp: 子集卷积指数	9
7. pow: 非负整数次幂与前导零	9
7.1 为什么不能直接对原数组移位	10
7.2 每个变换点分别处理前导零	10
7.3 大指数的三种处理	10
7.4 复杂度与调用	11
8. 通用复合: 固定新元素所在的块	11
8.1 把复合看成集合划分	11
8.2 为什么要维护权值偏移?	11
8.3 固定新元素所在的块	12
8.4 计算顺序与实现	12
8.5 复杂度为什么没有多一个 n ?	12
8.6 调用与来源	13
9. 完整实现 (打印用)	13
常用生成函数的推导补充: Catalan 与错排	20
1. Catalan 数: 函数方程、闭式与系数提取	20
1.1 二叉树分解得到函数方程	20
1.2 Lagrange 反演得到闭式	21
1.3 直接展开平方根, 提取 Catalan 系数	21
1.4 Catalan 生成函数的幂: 有序森林计数	22
1.5 反射原理与组合数差	22
1.6 Catalan 的应用识别	22
2. 错排数: 容斥与指数生成函数	22
2.1 按环分解: 推广到限制环长的排列	23
3. 使用时如何选择	23
Prüfer 序列: 标号树编码与计数	23
1. 编码: 反复删除最小叶子	23
2. 解码: 剩余出现次数决定当前叶子	23
3. 完整代码: 最小堆实现	24
4. Cayley 公式与指定度数	25
5. 指定叶子集合与恰有多少叶子	25
6. 带权 Cayley 与连接已有连通块	26

子集形式幂级数：inv、ln、exp、pow 与通用复合

完整代码附于第 9 节, 对应 sps.cpp 中的 sps_inv、sps_newton、sps_ln、sps_exp、sps_pow 和 sps_compose。以下运算均在模 $p = 998244353$ 的域中进行, 全集大小为 $n < p$ 。求逆要求 $f[0] \neq 0$; 其余操作的条件见后文。

本文使用 0-based 位掩码表示集合: 第 i 位表示元素 i 是否属于集合, 数组下标 s 就是集合 S 。输入数组长度必须为 $1 \ll n$, 并且所有系数先规范到 $[0, p)$ 。源文件末尾的 main 只是演示 sps_exp 的输入输出; 提交时通常删除它, 只保留前面的函数。

0. 可直接复制的接口

```
// sps.cpp 中已经包含以下接口; 将 sps.cpp 的 main 删除后即可作为模板使用。
vector<int> sps_inv(const vector<int>& f, int n);           // f[0] != 0
vector<int> sps_ln(const vector<int>& f, int n);           // f[0] == 1
vector<int> sps_exp(const vector<int>& f, int n);           // f[0] == 0
vector<int> sps_pow(const vector<int>& f, int n,
                    long long exponent);                   // exponent >= 0
vector<int> sps_compose(const vector<int>& outer,
                       const vector<int>& inner, int n); // inner[0] == 0
```

例如:

```
int n = 4;
vector<int> f(1 << n);
f[0] = 1;           // ln / inv 的常数项条件
f[1] = 2;
auto inverse = sps_inv(f, n);
auto logarithm = sps_ln(f, n);

vector<int> h(1 << n);
h[1] = 3;           // exp 的空集项必须为 0
auto exponential = sps_exp(h, n);
auto square = sps_pow(h, n, 2);
```

sps_inv 与 sps_newton 都是子集卷积逆; 前者按集合大小递推, 后者把每个 zeta 点看作普通 FPS 后作 Newton 迭代。两者答案相同, 但中间数组含义不同, 不能混用。完整实现见本文第 9 节, 也可使用 sps.cpp。

1. 定义与分层变换

子集卷积定义为

$$(f * g)[S] = \sum_{T \subseteq S} f[T]g[S \setminus T].$$

单位元 e 满足 $e[\emptyset] = 1$, 非空集合处为 0。求逆即求 g 使 $f * g = e$ 。

将 f 按集合大小分层, 再做子集 zeta 变换:

$$F_i[S] = \sum_{\substack{T \subseteq S \\ |T|=i}} f[T].$$

代码中的 $\text{fwt}(\dots, \text{false})$ 是子集 zeta 变换, $\text{fwt}(\dots, \text{true})$ 是其 Möbius 逆变换, 下面分别记作 $\mathcal{Z}$ 和 $\mathcal{M}$ 。

若 G_j 同样来自 g 的分层 zeta 变换, 则

$$C_k[S] = \sum_{i=0}^k F_i[S] G_{k-i}[S] = \sum_{\substack{A, B \subseteq S \\ |A|+|B|=k}} f[A]g[B].$$

按并集 $R = A \cup B$ 分组可见, C_k 是下面这个数组的 zeta 变换, 因此

$$\mathcal{M}(C_k)[S] = \sum_{\substack{A \cup B = S \\ |A|+|B|=k}} f[A]g[B].$$

当 $k = |S|$ 时, 由

$$|A| + |B| = |A \cup B| + |A \cap B|$$

得到 $A \cap B = \emptyset$, 所以

$$\mathcal{M}(C_{|S|})[S] = (f * g)[S].$$

这就是分层变换加速子集卷积的依据: 逆变换确定并集, 取集合大小对应的层排除重叠。

2. 按集合大小递推求逆

2.1 递推式的推导

将 $S = \emptyset$ 代入 $f * g = e$, 得到

$$f[\emptyset]g[\emptyset] = 1, \quad g[\emptyset] = f[\emptyset]^{-1}.$$

对于非空 S , 卷积结果应为 0。单独提出 $T = \emptyset$ 的项:

$$f[\emptyset]g[S] + \sum_{\emptyset \neq T \subseteq S} f[T]g[S \setminus T] = 0.$$

移项并除以 $f[\emptyset]$, 得到

$$\boxed{g[S] = -f[\emptyset]^{-1} \sum_{\emptyset \neq T \subseteq S} f[T]g[S \setminus T]}.$$

因为 $T \neq \emptyset$, 所以 $|S \setminus T| < |S|$ 。按集合大小递增计算, 右侧用到的值均已求出。这也证明了当且仅当 $f[\emptyset] \neq 0$ 时, 逆元存在且唯一。

直接对每个 S 枚举 T 的总复杂度为 $O(3^n)$ 。

2.2 用分层变换加速递推维护

$$G_j[S] = \sum_{\substack{T \subseteq S \\ |T|=j}} g[T].$$

初始时，所有位置的 $G_0[S]$ 都等于 $f[\emptyset]^{-1}$ 。计算第 k 层时， $G_0, \dots, G_{k-1}$ 已知，先计算

$$C_k[S] = \sum_{i=1}^k F_i[S] G_{k-i}[S].$$

这里 $i \geq 1$ 对应递推式中 T 非空。逆变换后，取 $|S| = k$ 的位置：

$$\begin{aligned} c_k[S] &= \mathcal{M}(C_k)[S] \\ &= \sum_{\substack{A \cup B = S \\ A \neq \emptyset \\ |A| + |B| = k}} f[A] g[B] \\ &= \sum_{\emptyset \neq A \subseteq S} f[A] g[S \setminus A]. \end{aligned}$$

于是

$$g[S] = -f[\emptyset]^{-1} c_k[S] \quad (|S| = k).$$

将这些 $g[S]$ 放入一个新数组，其余位置填 0，再做 zeta 变换，便得到后续需要的 G_k 。

对应 `sps_inv` 的流程：

1. 预处理 $F_1, \dots, F_n$ ，并初始化 $g[\emptyset]$ 和 G_0 。
2. 对 $k = 1, \dots, n$ ，计算 C_k 并做逆变换。
3. 只在 $|S| = k$ 处计算 $g[S]$ 。
4. 将第 k 层重新做 zeta 变换，得到 G_k 。

由于求和不使用 F_0 ，这个版本不需要变换 F_0 。

3. 分层 zeta + Newton 求逆

3.1 转化为逐点形式幂级数求逆

对每个集合 S ，定义普通形式幂级数

$$F_S(x) = \sum_{k=0}^n F_k[S] x^k.$$

其常数项恒为 $a = f[\emptyset] \neq 0$ ，因此可以计算

$$B_S(x) = F_S(x)^{-1} \pmod{x^{n+1}}.$$

记 $B_k[S] = [x^k] B_S(x)$ ，最终答案为

$$g[S] = \mathcal{M}(B_{|S|})[S].$$

代码 `sps_newton` 中用变量 `G` 保存这里的 B ；使用不同符号是为了区分它与递推版本的 G_k 。

为什么变换后做普通 FPS 运算，能得到子集幂级数运算？

先明确数组方向：固定集合 S ，把 $F_0[S], F_1[S], \dots, F_n[S]$ 当成一个 FPS 的系数。大小 k 是次数，不是独立做 FPS 的对象。运算结束后，才对每个大小层做逆变换，并取第 $|S|$ 层在 S 处的值。

整个方法只依赖一件事：普通的 j 次幂，经过“逆变换 + 取对应层”，会变成 j 次子集卷积。下面先证明它，再解释其他操作。

1. 变换后的多项式到底装了什么？

由定义，

$$F_S(x) = \sum_{T \subseteq S} f[T]x^{|T|}.$$

例如 $S = \{1, 2\}$ 时，

$$F_S(x) = f[\emptyset] + f[\{1\}]x + f[\{2\}]x + f[\{1, 2\}]x^2.$$

就是把 S 的每个子集对应的值，按这个子集的大小放到相应次数上。

2. 普通乘法比子集卷积多算了什么？

先看平方。展开 $F_S(x)^2$ ，就是从上面的和中选两项相乘：

$$f[A]x^{|A|} \cdot f[B]x^{|B|} = f[A]f[B]x^{|A|+|B|}, \quad A, B \subseteq S.$$

而我们想要的是

$$(f * f)[S] = \sum_{A \dot{\cup} B = S} f[A]f[B].$$

两者相差两个条件：普通乘法没有要求 $A \cup B = S$ ，也没有要求 $A \cap B = \emptyset$ 。

- 逆变换补上并集条件。固定次数，把普通乘法的各项按并集分组。 S 处的值等于“并集为 S 的各个子集”的组的总和，这正是 zeta 变换。因此逆变换恢复的，就是并集恰为 S 的那一组。
- 取第 $|S|$ 层补上不相交条件。并集已经是 S ；如果有元素重叠， $|A| + |B|$ 就会大于 $|S|$ 。所以只取次数 $|S|$ ，重叠项便被排除了。

例如目标为 $\{1, 2\}$ ：选择 $\{1\}, \{1\}$ 的项并集不够，会被逆变换排除；选择 $\{1\}, \{1, 2\}$ 的项次数为 3，不会进入第 2 层。

3. 为什么不只平方成立，而是任意次幂都成立？

展开 $F_S(x)^j$ 就是选 j 个子集，各自的值相乘，次数为它们的大小之和。仍然按并集分组做逆变换，再取次数 $|S|$ ：每个元素至少出现一次，总出现次数又只有 $|S|$ ，所以每个元素只能出现一次。

留下的恰好就是 j 个子集互不相交、并集为 S 的所有选择，也就是 $f^{*j}[S]$ 。这一步直接分析整个 j 次幂，中途不需要提取或清零。零次幂则对应单位元 e 。

4. `inv`、`ln`、`exp`、`pow` 都是幂的加权和。

令 $a = f[\emptyset]$ 、 $h = f - ae$ ，因此 $h[\emptyset] = 0$ 。把 h 做分层变换，在每个集合处得到一个常数项为零的普通 FPS，下面用 P 表示它。两边的展开逐项对应：

操作	变换后做普通 FPS 运算	最后提取出的子集运算
inv, $a \neq 0$	$(a + P)^{-1} = a^{-1} - a^{-2}P + a^{-3}P^2 - \dots$	$f^{*-1} = a^{-1}e - a^{-2}h + a^{-3}(h * h) - \dots$
ln, $a = 1$	$\ln(1+P) = P - P^2/2 + P^3/3 - \dots$	$\ln_* f = h - (h * h)/2 + h^{*3}/3 - \dots$
exp, $a = 0$	$\exp P = 1 + P + P^2/2! + \dots$	$\exp_* f = e + h + (h * h)/2! + \dots$
pow, 非负整数 t	直接算 $F_S(x)^t$	f^{*t}

原因是：每个普通幂都按第 3 步变成子集卷积幂，而逆变换和取对应层都保持加法与数乘。因此，整个展开式也对应。

h 的空集项为零，超过 n 个非空子集不可能互不相交，所以这些展开只需保留到 n 次。普通 FPS 中也只需保留到 x^n 。

Newton 或递推只是算出表格中普通 FPS 结果的方法；结果为什么能转回子集运算，由上面的幂次对应保证。

适用范围：是不是所有操作都能这样做？

不是所有操作。前面证明的是：只要操作能写成“子集卷积幂的加权和”，就可以转成逐点普通 FPS 运算。这就是普通形式幂级数与子集幂级数的复合。

先看空集项为零的 h 。给定普通形式幂级数

$$\Phi(z) = c_0 + c_1 z + c_2 z^2 + \dots,$$

把 z 换成 h ，并把普通乘法换成子集卷积，定义

$$\Phi_*(h) = c_0 e + c_1 h + c_2 (h * h) + \dots + c_n h^{*n}.$$

因为 $h^{*(n+1)} = 0$ ，后面全部为零，所以没有无限求和的问题，只需要 $c_0, \dots, c_n$ 。根据前面的幂次对应，变换后逐点计算 $\Phi(P)$ ，最后逆变换并提取，即得到 $\Phi_*(h)$ 。

对于一般的 $f = ae + h$ ，需要的是操作在常数 a 附近的展开：

$$\Phi(a + z) = c_0 + c_1 z + \dots + c_n z^n \pmod{z^{n+1}}.$$

只要这个展开有定义，且系数在当前模数下合法，就能把 z 换成 h ，使用同样的方法。例如求逆需要 $a \neq 0$ ； $\ln(1+z)$ 在 $a = 1$ 处展开；非负整数次幂本身是多项式，任何 a 都可以。

这里有两个边界：

- 普通形式幂级数不能随便代入非零常数。例如给定任意无穷级数 $\Phi(z) = \sum c_j z^j$ ，直接代入空集项为 1 的 f ，结果的空集项就需要计算 $\sum c_j$ ，在形式幂级数意义下通常没有定义。若是多项式，或像 $1/z$ 那样能在指定的非零 a 处另行展开，则可以。条件检查针对这个操作本身。
- 任意数组处理不一定是这种复合。例如“只保留下标集合含元素 1 的系数”依赖具体元素，不能由统一的标量系数 c_j 表示。以 $f = e$ 为例，结果要求 $c_0 = 0$ ；再分别令 f 只在 $\{1\}$ 或 $\{2\}$ 处为 1，两者都有 $f * f = 0$ ，任何这样的复合都只能给出 $c_1 f$ ，无法同时保留前者、删除后者。

本文在模 p 下且 $n < p$ ，因此 $\ln$ 、 $\exp$ 前 n 项中的 $1/k$ 、 $1/k!$ 均合法。其他操作若涉及分母、开根等，也要先确认所需系数存在；这个正确性结论并不保证任意复合都能在 $O(n^2 2^n)$ 内算完，具体复杂度取决于逐点 FPS 算法。

3.2 Newton 迭代式及精度倍增

以下固定一个 S ，省略下标。要求解

$$H(B) = FB - 1 = 0.$$

假设已有 $FB \equiv 1 \pmod{x^m}$ ，Newton 修正为

$$B_{\text{new}} = B - \frac{H(B)}{H'(B)} = B - \frac{FB - 1}{F}.$$

因为 $FB - 1$ 从 x^m 开始，而 $F^{-1} \equiv B \pmod{x^m}$ ，在模 x^{2m} 意义下可以用 B 代替修正项中的 F^{-1} ：

$$B_{\text{new}} = B(2 - FB) \pmod{x^{2m}}.$$

令误差 $E = 1 - FB$ ，则

$$1 - FB_{\text{new}} = 1 - FB(2 - FB) = (1 - FB)^2 = E^2.$$

原误差被 x^m 整除，新误差便被 x^{2m} 整除，所以精度倍增。初值为 $B = a^{-1} \pmod{x}$ 。

3.3 只计算新增的高半段

代码只保留已确定的前 m 项，将其余系数视为 0。设

$$K = \min(2m, n + 1), \quad h = K - m \leq m.$$

由于低 m 项的误差为 0，令

$$Q = \frac{FB - 1}{x^m} \pmod{x^h}, \quad J = F^{-1} \pmod{x^h}.$$

于是只需要

$$C = QJ \pmod{x^h}, \quad B_{\text{new}} = B - x^m C \pmod{x^K}.$$

因为 $h \leq m$ ，已有 B 的前 h 项就是 J ，不需要额外求逆。恢复集合下标后，高半段误差系数为

$$Q_t[S] = \sum_{j=0}^{m-1} F_{m+t-j}[S] B_j[S], \quad 0 \leq t < h.$$

常数项 -1 不影响这些次数。代码调用 `bag(Q, G, lim, h)` 计算 C ，并补上

$$B_{m+t}[S] = -C_t[S].$$

达到 $n + 1$ 项精度后，对各层做 Möbius 逆变换，提取 $|S| = k$ 的位置即可。

3.4 一般 Newton 形式

对允许上述形式 Taylor 展开的代数方程 $H(B) = 0$, 若 $H(B) = O(x^m)$ 且 $H'(B)$ 的常数项可逆, 则

$$B_{\text{new}} = B - H(B)(H'(B))^{-1} \pmod{x^K}.$$

修正量为 $O(x^m)$, Taylor 展开的二次及以上项为 $O(x^{2m})$, 因此得到倍增精度。仍可令

$$Q = \frac{H(B)}{x^m} \pmod{x^h}, \quad J = (H'(B))^{-1} \pmod{x^h},$$

然后只补高半段 $-x^m(QJ \bmod x^h)$ 。若方程含有对 x 求导等操作, 需要另外检查精度条件, 不能直接套用这一结论。

4. 两种实现的区别与复杂度

项目	sps_inv: 分层递推	sps_newton: Newton 迭代
中间数组	答案按大小分层后的 zeta 变换 G_k	普通幂级数逆的系数 B_k
推进方式	每次求出一个集合大小层	每次将幂级数精度倍增
提取集合大小对应层	每轮逆变换后提取, 再重新变换	全部迭代结束后提取
时间复杂度	$O(n^2 2^n)$	$O(n^2 2^n)$
空间复杂度	$O(n 2^n)$	$O(n 2^n)$

递推版本层间乘法共需 $O(n^2 2^n)$, 各层变换合计也为 $O(n^2 2^n)$ 。Newton 版本使用朴素层间卷积, 各轮的平方代价按倍增规模求和仍为 $O(n^2 2^n)$; 分层变换的代价相同。 $n = 0$ 时两者均为 $O(1)$ 。

Newton 迭代中不能按 $k = |S|$ 清零。例如 $S = \{v\}$, $F_S(x) = a + bx$, 则

$$B_S(x) = a^{-1} - ba^{-2}x + b^2a^{-3}x^2 - \dots$$

即使 $k > |S|$, 系数仍可能非零 (当所需截断精度包含该项时)。这些系数参与后续迭代, 必须保留到最后; 递推版本的中间数组则始终具有分层 zeta 的含义。

两种函数均要求 `f.size() == (1 << n)`、`0 <= f[S] < mod`、`f[0] != 0`, 且 `1 << n` 能用 `int` 表示。

5. ln: 子集卷积对数

5.1 定义及变换的正确性

要求 $f[0] = 1$ 。令 $u = f - e$, 定义

$$\ln_* f = \sum_{j=1}^n \frac{(-1)^{j+1}}{j} u^{*j}.$$

因为 $u[0] = 0$, 所以 $u^{*(n+1)} = 0$, 这个和是有限的, 结果的空集项为 0。

第 3.1 节对多个因子的分析同样适用: 在每个变换点求普通幂级数 $\ln F_S(x)$, 对各层做 Möbius 逆变换, 最后提取 $k = |S|$, 便得到 $\ln_* f$ 。逆变换筛出并集, 大小条件筛出两两不交; 因此普通幂级数的每个幂次都对应子集卷积的相同幂次。

5.2 普通幂级数递推的推导

固定一个变换点, 设 $A(x) = \sum a_k x^k$, $a_0 = 1$, $B(x) = \ln A(x) = \sum b_k x^k$, $b_0 = 0$ 。由 $B' = A'/A$, 得到 $AB' = A'$ 。比较 x^{k-1} 系数:

$$\sum_{i=1}^k i b_i a_{k-i} = k a_k.$$

提出 $i = k$ 的项, 利用 $a_0 = 1$:

$$b_k = a_k - \frac{1}{k} \sum_{i=1}^{k-1} i b_i a_{k-i}.$$

右侧只依赖 $b_1, \dots, b_{k-1}$, 可以顺序递推。这对应代码 `fps_ln`; `sps_ln` 负责分层变换、逐点调用和最终提取。这里的导数是对辅助变量 x 的普通形式求导。不能把子集数组的整数下标直接当成多项式次数。

6. exp: 子集卷积指数

要求 $f[\emptyset] = 0$, 定义

$$\exp_* f = \sum_{j=0}^n \frac{f^{*j}}{j!}, \quad f^{*0} = e.$$

结果的空集项为 1。同样可以逐点计算普通幂级数的指数, 最后逆变换并提取集合大小对应层。

固定一个变换点, 设 $a_0 = 0$, $B = \exp A$ 。由 $B' = A'B$, 比较 x^{k-1} 系数:

$$k b_k = \sum_{i=1}^k i a_i b_{k-i}.$$

所以递推式为

$$b_0 = 1, \quad b_k = \frac{1}{k} \sum_{i=1}^k i a_i b_{k-i}.$$

对应代码 `fps_exp` 和 `sps_exp`。所有 $1/k$ 都是模逆元, 由 `small_inverses` 预处理; 要求 $n < p$, 以保证这些分母可逆。

这两种实现都直接使用逐点递推, 时间为 $O(n^2 2^n)$ 、空间为 $O(n 2^n)$, 中途保留全部层系数。满足常数项条件时, 有

$$\exp_*(\ln_* f) = f, \quad \ln_*(\exp_* f) = f.$$

7. pow: 非负整数次幂与前导零

接口为 `sps_pow(f, n, t)`, 其中 `t` 是非负 `long long`, 返回 f^{*t} 。约定任何数组的零次幂都是 e , 包括全零数组。

7.1 为什么不能直接对原数组移位

子集数组下标代表集合，乘法要求集合互不相交。它不是以下标整数加法为次数的普通多项式，因此不能找到第一个非零数组元素后直接平移下标、除去一个“首项”。

可以先做一个全局判零：若 f 非零，令

$$r = \min_{f[T] \neq 0} |T|.$$

若 $r > 0$ ， t 个非零因子对应的互不相交子集至少占用 rt 个元素，所以 $rt > n$ 时必有 $f^{*t} = 0$ 。代码使用 $\mathbf{t} > \mathbf{n} / \mathbf{r}$ 判断，避免乘法溢出。 $rt \leq n$ 只是可能非零，并不保证非零，因为还可能重叠或相消。

7.2 每个变换点分别处理前导零

分层 zeta 后，在一个固定点写

$$A(x) = x^d c H(x), \quad c = a_d \neq 0, \quad H(0) = 1,$$

其中 d 是该点首个非零系数的次数。不同点可能有不同的 d ，甚至整个幂级数都为零；必须逐点寻找。

对 $t > 0$ ，若该点全零或 $dt > n$ ，该点的输出全部为零。否则

$$A(x)^t = x^{dt} c^t \exp(t \ln H(x)) \pmod{x^{n+1}}.$$

证明是先分离 x^d 和非零常数 c ，再对常数项为 1 的 H 使用 $H^t = \exp(t \ln H)$ 。这里只需保留到次数 $n < p$ ，涉及的整数分母均可逆。

令 $L = n - dt$ ，只需要 H 的前 $L + 1$ 项：

$$h_k = a_{d+k} c^{-1} \quad (0 \leq k \leq L).$$

由于 $t \geq 1$ ，有 $d + L \leq n$ ，输入系数足够。计算 $B = \exp(t \ln H) \bmod x^{L+1}$ 后，将 $c^t b_k$ 写入第 $k + dt$ 层。最后统一逆变换并提取对角层，得到子集卷积的幂。即使普通幂级数中的某些乘积来自重复使用同一个子集，它们也会在最后的并集与总大小筛选中被排除。

7.3 大指数的三种处理

用途	使用的指数	原因
判断 $dt > n$ 和计算位移	原始 t	次数不能取模；先用 $\mathbf{t} > \mathbf{n} / \mathbf{d}$ 排除超界
计算 $t \ln H$	$t \bmod p$	系数在模 p 的域中运算
计算非零常数 c^t	$t \bmod (p - 1)$	费马小定理，且已确保 $c \neq 0$

不能先把整个指数对 p 或 $p - 1$ 取模，再进行前导零判断。

例如 $f[\{1\}] = a$ 、 $f[\{2\}] = b$ ，其余为零，则

$$f^{*2}[\{1, 2\}] = 2ab, \quad f^{*3} = 0.$$

若只有 $f[\{1\}] \neq 0$ ，则平方已经为零，因为同一个非空子集不能与自身不相交。

7.4 复杂度与调用

当前 `pow` 每点调用一次普通 `ln` 和 `exp`，并计算首项的逆元与幂。时间为 $O(n^2 2^n + 2^n \log p)$ ，空间为 $O(n 2^n)$ ；提前判零时仅需扫描输入。模数固定时，快速幂部分每点为固定数量的模乘。

```
auto a = sps_ln(f, n);           // f[0] == 1
auto b = sps_exp(f, n);          // f[0] == 0
auto c = sps_pow(f, n, 5LL);     // f[0] 可以为零
auto e = sps_pow(f, n, 0LL);     // 单位元
```

上述接口均要求输入长度为 $1 \ll n$ ，系数位于 $[0, \text{mod})$ ，且 $1 \ll n$ 能用 `int` 表示。`pow` 当前只接受非负整数指数；新增接口用断言检查常数项或指数条件。当前 `main` 调用 `sps_exp`，使用其他运算时替换调用即可。

8. 通用复合：固定新元素所在的块

接口为 `sps_compose(a, f, n)`，计算

$$A_*(f) = a_0 e + a_1 f + a_2 (f * f) + \dots + a_n f^{*n}, \quad A(x) = \sum_{k \geq 0} a_k x^k.$$

要求 $f[0] = 0$ 。因此 $f^{*(n+1)} = 0$ ，外层只用前 $n+1$ 项；缺项补零，多余项忽略。`a` 传普通系数 a_k ，不需要调用者乘阶乘；空 `a` 表示零多项式。

本节的通用算法也是 $O(n^2 2^n)$ 。第 3.1 节解释了为什么可以转成逐点普通 FPS 复合，但如果逐点使用朴素 Horner 法，时间是 $O(n^3 2^n)$ 。下面直接利用集合划分的结构达到更好的复杂度。

8.1 把复合看成集合划分

$f^{*k}[S]$ 枚举把 S 拆成 k 个有序非空块的方案，权值为各块的 f 值之积。

同一个无序划分的 k 个块有 $k!$ 种排列，所以令

$$b_k = k! a_k,$$

复合就等价于：枚举无序集合划分；若分成 k 块，就把所有块的 f 值相乘，再乘 b_k 。

例如 $S = \{1, 2\}$ 只有“整块”和“两个单点”两种划分：

$$A_*(f)[\{1, 2\}] = b_1 f[\{1, 2\}] + b_2 f[\{1\}] f[\{2\}].$$

8.2 为什么要维护权值偏移？

先选定一个块，再划分剩余元素。如果剩余部分分成 k 块，总共就是 $k+1$ 块，应该使用 b_{k+1} ，不能继续使用 b_k 。

因此定义 $H_r[S]$ ：划分 S ，若分成 k 块，使用权值 b_{r+k} 。每选好一块，权值偏移 r 就加一。

没有元素时，只有零个块的空划分，块权值之积为 1，所以

$$H_r[\emptyset] = b_r.$$

最终答案是 H_0 。代码中的 `H[r][s]` 保存这个状态：`r` 不是集合大小层。

8.3 固定新元素所在的块

假设已处理前 m 个元素，现在加入新元素 v 。对于含 v 的集合 $S \cup \{v\}$ ，每种划分中，含 v 的块唯一。设这个块为 $T \cup \{v\}$ ，其余元素就是 $S \setminus T$ 。

目标集合为 $\{1, 2, 3\}$ ，新元素为 3	剩余需要划分的集合
选定块 $\{3\}$	$\{1, 2\}$
选定块 $\{1, 3\}$	$\{2\}$
选定块 $\{2, 3\}$	$\{1\}$
选定块 $\{1, 2, 3\}$	空集

选定的块贡献 $f[T \cup \{v\}]$ ，剩余部分使用偏移 $r+1$ 。因此

$$H_r^{\text{new}}[S \cup \{v\}] = \sum_{T \subseteq S} f[T \cup \{v\}] H_{r+1}^{\text{old}}[S \setminus T].$$

令 $q[T] = f[T \cup \{v\}]$ ，右侧恰好就是子集卷积 $(q * H_{r+1}^{\text{old}})[S]$ 。对于不含 v 的位置，直接保留旧值。

在位掩码中，新元素对应 $\text{lim} = 1 \ll m$ ，所以 q 就是输入的一段 $f[\text{lim} \dots 2 \cdot \text{lim}]$ 。去掉新元素后， $q[0]$ 表示单点块的权值，可以非零。

8.4 计算顺序与实现

开始只有空集，维护 $H_0, \dots, H_n$ 。加入一个元素后，只需维护 $H_0, \dots, H_{n-1}$ ；全部加入后，只剩答案 H_0 。

这是因为新一轮的 H_r 只依赖旧一轮的 H_r 和 H_{r+1} 。从最终的 H_0 往回看，剩下多少轮，就只需要额外保留多少个偏移。

```

H[r] = { r! * a[r] }           // r = 0 .. n
for m = 0 .. n-1:
    q = f[2^m .. 2^(m+1))
    Q = 分层 zeta(q)           // 这一轮只计算一次
    for r = 0 .. n-m-1:       // 必须从小到大
        high = 子集卷积 (q, H[r+1]) // 复用 Q
        在 H[r] 后面接上 high
    删除最后一个 H
return H[0]

```

$H[r]$ 的旧内容作为低半段保留，新算出的卷积作为高半段。按 r 递增更新时， $H[r+1]$ 尚未修改，读取的就是上一轮数据。反过来更新则会错误地读取新一轮数据。

8.5 复杂度为什么没有多一个 n ?

从 m 个元素扩展到 $m+1$ 个元素，做 $n-m$ 次规模为 2^m 的子集卷积，因此时间为

$$\sum_{m=0}^{n-1} (n-m) O((m+1)^2 2^m).$$

令 $d = n - m$ ，利用 $(m+1)^2 \leq n^2$ ，得到

$$\sum_{m=0}^{n-1} (n-m)(m+1)^2 2^m \leq n^2 2^n \sum_{d=1}^n \frac{d}{2^d} < 2n^2 2^n.$$

所以总时间为 $O(n^2 2^n)$ 。需要维护的偏移多时，集合规模小；集合规模大时，偏移数量已经很少。

代码只保留当前轮的 H 和当前卷积的分层数组，空间为 $O(n 2^n)$ 。 $n = 0$ 时直接得到只含 a_0 的数组，时间、空间均为 $O(1)$ 。

8.6 调用与来源

```
vector<int> a = {3, 5, 7};
auto g = sps_compose(a, f, n); //  $3e + 5f + 7(f*f)$ , 要求  $f[0] == 0$ 
```

内层 `f.size()` 必须等于 $1 \ll n$ ；两组输入系数均在 $[0, \text{mod})$ 。该接口不直接接受非零空集项；这种情况需要先把外层在该常数处展开，再对去掉空集项的内层调用。

来源：李白天 (Elegia) 2021 年国家集训队论文《信息学竞赛中的生成函数计算理论框架》，“逐点牛顿迭代法 $\rightarrow$ 复合”。上面的划分递推与论文的导数递推等价： H_r 就是 $A^{(r)}$ 复合当前内层的结果。

- 作者的论文源码
- 作者的算法介绍

9. 完整实现（打印用）

以下代码与 `sps.cpp` 一致，包含全部辅助函数、两种求逆、`ln`、`exp`、`pow`、复合和演示 `main`。复用模板时删除 `main`；输入条件与复杂度见前文。

```
#include <iostream>
#include <algorithm>
#include <vector>
#include <cassert>
using namespace std;

const int mod = 998244353;

int A(int x, int y){ return x + y >= mod ? x + y - mod : x + y; }
int S(int x, int y){ return x - y < 0 ? x + mod - y : x - y; }
int N(int x){ return x ? mod - x : 0; }

void fwt(vector<int>& a, int lim, bool inv){
    for(int i = 1; i < lim; i <= 1)
        for(int j = 0; j < lim; j += i < 1)
            for(int k = 0; k < i; k++){
                int x = a[j + k], y = a[j + k + i];
                a[j + k + i] = !inv ? A(x, y) : S(y, x);
            }
}

using FPS = vector<vector<int>>;

// FWT 意义下逐点卷积，结果截断到前 K 项；lim 是点数
FPS bag(const FPS& a, const FPS& b, int lim, int K){
    FPS c(K, vector<int>(lim));
    int na = min((int)a.size(), K);
    int nb = min((int)b.size(), K);
    for(int i = 0; i < na; i++){
        for(int j = 0; j < nb && i + j < K; j++){
            const auto &x = a[i];
            const auto &y = b[j];
```

```

        auto &z=c[i+j];
        for(int s=0;s<lim;s++)
            z[s]=(z[s]+1ll*x[s]*y[s])%mod;
    }
    return c;
}

```

```

int _pow(int a, int b){
    int r = 1;
    while(b){
        if(b & 1) r = 1LL * r * a % mod;
        a = 1LL * a * a % mod;
        b >>= 1;
    }
    return r;
}

```

```

#define pc(x) __builtin_popcount(x)

```

/*

子集卷积求逆（按集合大小递推）：

1. 定义 $(f*g)[S] = \sum_{T \text{ subseq } S} f[T]g[S \setminus T]$ ，要求 $f*g=e$ ，其中 $e[\text{空集}]=1$ ，其余为 0 。代入 $S=\text{空集}$ ，得

$f[0]g[0]=1$ ，即 $g[0]=f[0]^{-1}$ ，因此要求 $f[0] \neq 0 \pmod{\text{mod}}$ 。

对非空 S ，把 $T=\text{空集}$ 这一项单独提出：

$f[0]g[S] + \sum_{\text{非空 } T \text{ subseq } S} f[T]g[S \setminus T] = 0$ ，

所以递推式为

$g[S] = -f[0]^{-1} * \sum_{\text{非空 } T \text{ subseq } S} f[T]g[S \setminus T]$ 。

因为 T 非空，所以 $|S \setminus T| < |S|$ ；按 $|S|$ 从小到大计算即可，且解唯一。

直接枚举 S, T 需要 $O(3^n)$ ，下面用分层 zeta 加速每一层的求和。

2. 记 $F_i[S] = \sum_{T \text{ subseq } S, |T|=i} f[T]$ ，

$G_j[S] = \sum_{T \text{ subseq } S, |T|=j} g[T]$ 。

计算第 k 层时， $G_0, \dots, G_{k-1}$ 已知。先在变换域计算

$C_k[S] = \sum_{i=1}^k F_i[S]G_{k-i}[S]$ 。

展开后，它枚举 $A, B \text{ subseq } S$ ，满足 A 非空、 $|A|+|B|=k$ 。

对 C_k 做 Möbius 逆变换，把“并集包含于 S ”变成“并集恰为 S ”：

$c[S] = \sum_{A \text{ union } B=S, A \text{ 非空}, |A|+|B|=k} f[A]g[B]$ 。

只取 $|S|=k$ 的位置，由 $|A|+|B|=|A \text{ union } B|+|A \text{ intersect } B|$

可知 $A \text{ intersect } B = \text{空集}$ ，于是 $B=S \setminus A$ ，恰好得到递推式中的求和。

因而代码中 $g[S] = -c[S] * \text{inv}f_0$ ，再把这些值放入第 k 层并做 zeta，

得到 G_k ，供后续各层使用。 i 从 1 开始，故不需要变换 F_0 。

3. 这里的 G_k 始终是“仅保留 $|S|=k$ 的 g 后做 zeta”的结果，每轮都先逆变换、提取第 k 层，再重新变换；与下方 Newton 版本直接维护逐点普通幂级数的逆不同，不能混用两者的中间层含义。

复杂度：时间 $O(n^2 \cdot 2^n)$ ，空间 $O(n \cdot 2^n)$ ($n=0$ 时为 $O(1)$)。

*/

```

vector<int> sps_inv(const vector<int>& f, int n){
    int lim = 1 << n;
    FPS F(n + 1, vector<int>(lim));
    FPS G(n + 1, vector<int>(lim));
    for(int s = 0; s < lim; s++)
        F[pc(s)][s] = f[s];
    // F[0] 后面不会参与卷积，不需要变换
    for(int i = 1; i <= n; i++)

```

```

        fwt(F[i], lim, false);
    int invf0 = _pow(f[0], mod - 2);
    vector<int> g(lim);
    g[0] = invf0;
    // zeta({g[0], 0, 0, ...}) 后所有位置都是 g[0]
    fill(G[0].begin(), G[0].end(), invf0);
    for(int k = 1; k <= n; k++){
        vector<int> c(lim);
        for(int i = 1; i <= k; i++)
            for(int s = 0; s < lim; s++)
                c[s] = (c[s] + 1ll * G[k-i][s] * F[i][s]) % mod;
        fwt(c, lim, true);
        for(int s = 0; s < lim; s++){
            if(pc(s) != k) continue;
            g[s] = 1ll * N(c[s]) * invf0 % mod;
            G[k][s] = g[s];
        }
        fwt(G[k], lim, false);
    }

    return g;
}

// 分层 zeta / 最后逆变换并提取对角层。
FPS sps_zeta(const vector<int>& f, int n){
    int lim = 1 << n;
    assert((int)f.size() == lim);
    FPS F(n + 1, vector<int>(lim));
    for(int s = 0; s < lim; s++) F[pc(s)][s] = f[s];
    for(auto &a : F) fwt(a, lim, false);
    return F;
}

vector<int> sps_extract(FPS& F, int n){
    int lim = 1 << n;
    vector<int> g(lim);
    for(int k = 0; k <= n; k++){
        fwt(F[k], lim, true);
        for(int s = 0; s < lim; s++)
            if(pc(s) == k) g[s] = F[k][s];
    }
    return g;
}

vector<int> small_inverses(int n){
    vector<int> inv(n + 1);
    if(n) inv[1] = 1;
    for(int i = 2; i <= n; i++)
        inv[i] = mod - 1LL * (mod / i) * inv[mod % i] % mod;
    return inv;
}

```

// 普通 FPS: $a[0]=1$, $b=\ln(a)$ 。由 $a*b'=a'$ 比较 $x^{(k-1)}$:

```

//  $b[k]=a[k]-1/k * \sum_{i=1}^{k-1} i*b[i]*a[k-i]$ 。
vector<int> fps_ln(const vector<int>& a, const vector<int>& inv){
    assert(a[0] == 1);
    int K = (int)a.size();
    vector<int> b(K);
    for(int k = 1; k < K; k++){
        int sum = 0;
        for(int i = 1; i < k; i++)
            sum = (sum + 1LL * i * b[i] % mod * a[k-i]) % mod;
        b[k] = S(a[k], 1LL * sum * inv[k] % mod);
    }
    return b;
}

// 普通 FPS:  $a[0]=0, b=\exp(a)$ 。由  $b'=a'*b$ :
//  $b[0]=1, b[k]=1/k * \sum_{i=1}^k i*a[i]*b[k-i]$ 。
vector<int> fps_exp(const vector<int>& a, const vector<int>& inv){
    assert(a[0] == 0);
    int K = (int)a.size();
    vector<int> b(K);
    b[0] = 1;
    for(int k = 1; k < K; k++){
        int sum = 0;
        for(int i = 1; i <= k; i++)
            sum = (sum + 1LL * i * a[i] % mod * b[k-i]) % mod;
        b[k] = 1LL * sum * inv[k] % mod;
    }
    return b;
}

// 子集卷积 Ln:  $f[0]=1$ 。逐点求普通 FPS 的 Ln, 最后提取对角层。
vector<int> sps_ln(const vector<int>& f, int n){
    assert(!f.empty() && f[0] == 1);
    FPS F = sps_zeta(f, n);
    auto inv = small_inverses(n);
    for(int s = 0; s < (1 << n); s++){
        vector<int> a(n + 1);
        for(int k = 0; k <= n; k++) a[k] = F[k][s];
        auto b = fps_ln(a, inv);
        for(int k = 0; k <= n; k++) F[k][s] = b[k];
    }
    return sps_extract(F, n);
}

// 子集卷积 exp:  $f[0]=0$ 。所有非对角系数都保留到计算结束。
vector<int> sps_exp(const vector<int>& f, int n){
    assert(!f.empty() && f[0] == 0);
    FPS F = sps_zeta(f, n);
    auto inv = small_inverses(n);
    for(int s = 0; s < (1 << n); s++){
        vector<int> a(n + 1);
        for(int k = 0; k <= n; k++) a[k] = F[k][s];
        auto b = fps_exp(a, inv);
    }
}

```

```

        for(int k = 0; k <= n; k++) F[k][s] = b[k];
    }
    return sps_extract(F, n);
}

/*
子集卷积非负整数次幂, 约定  $f^{\{0\}}=e$  (包括  $f$  全零)。
先按最低非零集合大小  $r$  判断:  $r>0$  且  $t>n/r$  时结果为零。
对每个变换点单独找普通 FPS 的首个非零项  $a[d]$ :
 $A(x)=x^d*c*H(x)$ ,  $H(0)=1$ ,  $A(x)^t=x^{(d*t)}*c^t*exp(t*ln H(x))$ 。
若该点全零, 或  $d>0$  且  $t>n/d$ , 则该点结果全零。
注意不能在原子集数组中按下标平移; 只在变换后的普通 FPS 中移位。
位移使用原始  $t$ ,  $ln$  的倍数用  $t\%mod$ , 非零  $c$  的幂指数用  $t\%(mod-1)$ 。
时间  $O(n^2*2^n + 2^n*Log(mod))$ , 空间  $O(n*2^n)$ 。
*/
vector<int> sps_pow(const vector<int>& f,int n,long long t){
    assert(t >= 0 && (int)f.size() == (1 << n));
    int lim = 1 << n;
    vector<int> zero(lim);
    if(t == 0){ zero[0] = 1; return zero; }
    int r = n + 1;
    for(int s = 0; s < lim; s++){
        if(f[s]) r = min(r, pc(s));
    }
    if(r == n + 1 || (r > 0 && t > n / r)) return zero;
    FPS F = sps_zeta(f, n);
    auto inv = small_inverses(n);
    for(int s = 0; s < lim; s++){
        vector<int> a(n + 1);
        for(int k = 0; k <= n; k++){
            a[k] = F[k][s];
            F[k][s] = 0;
        }
        int d = 0;
        while(d <= n && a[d] == 0) d++;
        if(d > n || (d > 0 && t > n / d)) continue;
        int shift = d ? (int)(d * t) : 0;
        int K = n - shift + 1;
        int ic = _pow(a[d], mod - 2);
        int scale = _pow(a[d], (int)(t % (mod - 1)));
        vector<int> h(K);
        for(int k = 0; k < K; k++) h[k] = 1LL * a[d+k] * ic % mod;
        auto b = fps_ln(h, inv);
        for(int &x : b) x = 1LL * x * (t % mod) % mod;
        b = fps_exp(b, inv);
        for(int k = 0; k < K; k++) F[k+shift][s] = 1LL * b[k] * scale % mod;
    }
    return sps_extract(F, n);
}

/*
普通 FPS 复合集合幂级数:  $a(x)=\sum a[k]*x^k$ , 返回  $\sum a[k]*f^{\{k\}}$ 。
要求  $f[0]=0$ ;  $a$  是普通系数, 不是 EGF 系数。缺项补零, 超过  $n$  次忽略。
*/

```

推导（固定新元素所在的块）：

1. $f^{\wedge\{*k\}}$ 枚举 k 个有序非空块；每个无序划分恰好出现 $k!$ 次。
令 $b[k]=k!*a[k]$ ，复合就是：枚举集合划分，分成 k 块时乘 $b[k]$ ，再乘所有块的 f 值。
2. $H[r][S]$ 表示划分 S ，分成 k 块时使用权值 $b[r+k]$ 的结果。
没有元素时只有空划分，所以 $H[r][\emptyset]=b[r]$ 。最终要求 $H[0]$ 。
3. 已处理低 m 位，加入新元素 v （对应位 $\text{lim}=1\ll m$ ）。
不含 v 的位置不变。含 v 时，其所在的块唯一，可写成 $T \text{ union } \{v\}$ ；
剩下 $S\setminus T$ 继续划分，已经选了一块，故权值偏移从 r 变成 $r+1$ ：

$$H_{\text{new}}[r][S \text{ union } \{v\}] = \sum_{T \subseteq S} f[T \text{ union } \{v\}] * H_{\text{old}}[r+1][S\setminus T].$$
令 $q[T]=f[T \text{ union } \{v\}]$ ，右侧就是 $q * H_{\text{old}}[r+1]$ 的子集卷积。
 q 去掉了 v ，因此 $q[\emptyset]=f[\{v\}]$ 可以非零。每轮只变换一次 q 。
4. 加入 m 位后仅保留 $r=\emptyset..n-m$ ；下一轮计算 $r=\emptyset..n-m-1$ 。
 r 从小到大更新，读取 $H[r+1]$ 时它仍是旧数组，卷积结果接到 $H[r]$ 后。

每轮做 $n-m$ 次 m 元子集卷积，时间

$$\sum_{m=\emptyset}^{n-1} (n-m) * O((m+1)^2 * 2^m) = O(n^2 * 2^n).$$

因为令 $d=n-m$ ， $(n-m)*2^m=2^n*d/2^d$ ，而 $\sum d/2^d$ 收敛。

空间 $O(n*2^n)$ ， $n=\emptyset$ 时 $O(1)$ 。只保留当前轮的 H ，复用 q 的变换。

来源：李白天《信息学竞赛中的生成函数计算理论框架》，“逐点牛顿迭代法 / 复合”。

<https://codeforces.com/blog/entry/92183>

```

*/
vector<int> sps_compose(const vector<int>& a, const vector<int>& f, int n){
    assert(n >= 0 && n < 31);
    assert((int)f.size() == (1 << n) && f[0] == 0);
    if(a.empty()) return vector<int>(1 << n);
    // H 的第一维是权值偏移 r，不是分层 zeta 的大小层。
    vector<vector<int>> H(n + 1, vector<int>(1));
    int fac = 1;
    for(int r = 0; r <= n; r++){
        if(r) fac = 1LL * fac * r % mod;
        if(r < (int)a.size()) H[r][0] = 1LL * fac * a[r] % mod;
    }
    for(int m = 0; m < n; m++){
        int lim = 1 << m;
        vector<int> q(f.begin() + lim, f.begin() + (lim << 1));
        FPS Q = sps_zeta(q, m);
        for(int r = 0; r < n - m; r++){
            FPS R = sps_zeta(H[r+1], m);
            R = bag(Q, R, lim, m + 1);
            auto high = sps_extract(R, m);
            // 低半段继承旧值，高半段存含新元素的结果。
            H[r].insert(H[r].end(), high.begin(), high.end());
        }
        H.pop_back();
    }
    return H[0];
}

```

/*

子集卷积求逆（分层 zeta + Newton）：

1. 定义 $(f*g)[S] = \sum_{T \subseteq S} f[T]g[S\setminus T]$ ，单位元为 $e[\text{空集}]=1$ ，

其余为 0。要求 $f*g=e$ ；当且仅当 $f[0] \neq 0$ 时逆元存在且唯一。

2. 令 $F_k[S] = \sum_{T \subseteq S, |T|=k} f[T]$ ，即按 $|T|$ 分层后做 *zeta*。

对每个 S ，把 $F[S](x) = \sum_k F_k[S]x^k$ 当作普通形式幂级数。

层间乘法用 *bag* 完成；对乘积的第 k 层做 Möbius 逆变换，得到

$$\sum_{A \cup B = S, |A|+|B|=k} f[A]g[B].$$

取 $k=|S|$ 时 A, B 必不相交，恰好得到子集卷积。

因此可逐点求 $G[S](x) = 1/F[S](x) \bmod x^{n+1}$ ，最后逆变换并取

$$g[S] = G_{|S|}[S].$$

也可从 $1/F = f[0]^{-1} \sum_{j \geq 0} (-U/f[0])^j$

理解： $U = F - f[0]$ 的每个因子代表非空子集，逆变换筛选并集为 S ，

总大小等于 $|S|$ 又筛掉所有重叠，留下子集卷积逆元的展开。

3. 一般 Newton：若 $H(G) = 0 \bmod x^m$ ，且 $H'(G)$ 的常数项可逆，则

$$G_{\text{new}} = G - H(G)/H'(G) \bmod x^K, K = \min(2m, n+1).$$

修正量从 x^m 开始，Taylor 展开的二次及以上项均为 $O(x^{2m})$ ，

故精度倍增。令 $h = K - m$ ， $Q = (H(G)/x^m) \bmod x^h$ ，

$$J = 1/H'(G) \bmod x^h, \text{ 只需 } C = Q * J \bmod x^h, \text{ 再补 } G[m+t] = -C[t].$$

这里 G 原来仅有前 m 项，计算 $H(G)$ 时其余项视为 0。

4. 求逆套用 $H(G) = F * G - 1$ ， $H'(G) = F$ 。初值 $G_0[S] = 1/f[0]$ ；

因 $h \leq m$ ，已有 G 的前 h 项就是所需 J ，无须另写 *inv_dH*。

高半段 $Q_t[S] = \sum_{i+j=m+t, j < m} F_i[S]G_j[S]$ ，然后用 *bag*(Q, G)。

若套用其他方程，替换初值、 Q 和 J 的计算，并先确认导数可逆；

若 H 含求导等操作，还需单独检查上述精度倍增条件是否成立。

注意：迭代在逐点普通幂级数中进行，中途不能按 $k=|S|$ 清零；

$G_k[S]$ 在 $k > |S|$ 时也可能非零，必须保留，最后才提取对角层。

用法：auto $g = \text{sps_newton}(f, n)$ ； $f.size() == 2^n$ ， $0 \leq f[S] < \text{mod}$ ， $f[0] \neq 0$ ；

$n > 0$ 且 $1 < n$ 可用 *int* 表示。返回 g ，使 $f * g = e$ (模 mod)。

复杂度：时间 $O(n^2 \cdot 2^n)$ ，空间 $O(n \cdot 2^n)$ ($n=0$ 时为 $O(1)$)。

分层变换参考：<https://arxiv.org/abs/cs/0611101>

*/

```
vector<int> sps_newton(const vector<int>& f, int n){
    int lim = 1 << n;
    FPS F(n + 1, vector<int>(lim));
    for(int s = 0; s < lim; s++)
        F[pc(s)][s] = f[s];
    for(int k = 0; k <= n; k++)
        fwt(F[k], lim, false);

    FPS G(1, vector<int>(lim, _pow(f[0], mod - 2)));
    for(int m = 1; m <= n; m <= 1){
        int K = min(n + 1, m << 1), h = K - m;
        // Q = [x^m, ..., x^(K-1)](F * G - 1), 常数 -1 不影响高半段。
        FPS Q(h, vector<int>(lim));
        for(int t = 0; t < h; t++)
            for(int j = 0; j < m; j++)
                for(int s = 0; s < lim; s++)
                    Q[t][s] = (Q[t][s] + 1LL * F[m+t-j][s] * G[j][s]) % mod;

        // J = 1/F mod x^h = G mod x^h; bag 自动截断到 h 项。
        FPS C = bag(Q, G, lim, h);
        G.resize(K, vector<int>(lim));
        for(int t = 0; t < h; t++)
            for(int s = 0; s < lim; s++)
                G[m+t][s] = N(C[t][s]);
    }
}
```

```

vector<int> g(lim);
for(int k = 0; k <= n; k++){
    fwt(G[k], lim, true);
    for(int s = 0; s < lim; s++)
        if(pc(s) == k) g[s] = G[k][s];
}
return g;
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n; cin >> n;
    vector<int> ar((1 << n));
    for(int i = 0; i < (1 << n); i++)
        cin >> ar[i];
    auto g = sps_exp(ar, n);
    for(int i = 0; i < (1 << n); i++)
        cout << g[i] << ' ';
    cout << '\n';
    return 0;
}

```

常用生成函数的推导补充：Catalan 与错排

本节作为 `math` 的补充，集中保留 Catalan 的代数生成函数与系数提取、错排的 EGF 与环长限制。基础 OGF/EGF、递推转有理函数、前缀和、Stirling/Bell 以及整数分拆查阅主目录，避免打印时重复收录速查公式。

- 基础建模与操作：生成函数基础。
- 公式、Stirling/Bell 与分拆：计数公式速查、Stirling 数。
- 求系数实现：线性递推、多项式与 FPS。

默认 $[x^n]F(x)$ 表示 x^n 的系数；EGF 提取系数后还要乘 $n!$ 。推导在有理数形式幂级数环中进行，不要求解析收敛。模数计算时，实际使用的 n 、 $n+1$ 、阶乘等分母必须可逆；整数闭式可整除不代表能够直接模除。

1. Catalan 数：函数方程、闭式与系数提取

记

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n-1}.$$

Catalan 数统计括号匹配、二叉树、凸多边形三角剖分、以及不穿过对角线的格路。

1.1 二叉树分解得到函数方程

令 $C(x) = \sum_{n \geq 0} C_n x^n$ 。这里使用满有序二叉树，以内部结点数为规模；单个叶子是规模为零的对象。一棵非平凡树由内部根、左子树和右子树组成。若左子树含 i 个内部结点，右子树含 $n-1-i$ 个，枚举根的左右子树得到

$$C_0 = 1, \quad C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}.$$

乘以 x^n 求和:

$$C(x) = 1 + xC(x)^2.$$

解二次方程并选择常数项为 1 的分支:

$$C(x) = \frac{1 - \sqrt{1 - 4x}}{2x}.$$

有理化也可写成 $C(x) = 2/(1 + \sqrt{1 - 4x})$, 避免在 $x = 0$ 处误以为有奇点。

1.2 Lagrange 反演得到闭式

令 $T(x) = C(x) - 1$, 函数方程变成

$$T = x(1 + T)^2.$$

Lagrange 反演: 若 $Y = x\Phi(Y)$, 则

$$[x^n]Y(x)^k = \frac{k}{n}[u^{n-k}]\Phi(u)^n.$$

取 $k = 1$ 、 $\Phi(u) = (1 + u)^2$ 。对 $n \geq 1$, 有 $C_n = [x^n]T$, 所以

$$C_n = \frac{1}{n}[u^{n-1}](1 + u)^{2n} = \frac{1}{n} \binom{2n}{n-1} = \frac{1}{n+1} \binom{2n}{n}.$$

函数方程直接比较系数得到卷积递推; Lagrange 反演进一步把它化成闭式:

$$C_n = \frac{1}{n+1} \binom{2n}{n}.$$

1.3 直接展开平方根, 提取 Catalan 系数

广义二项式定理给出

$$\sqrt{1 - 4x} = \sum_{j \geq 0} \binom{1/2}{j} (-4x)^j.$$

分子常数项抵消, 除以 x 对应下标移动一位。因此对 $n \geq 0$,

$$[x^n]C(x) = -\frac{1}{2}(-4)^{n+1} \binom{1/2}{n+1} = \frac{(2n)!}{n!(n+1)!} = C_n.$$

中间等式可把 $\binom{1/2}{n+1}$ 的分子展开为 $\frac{1}{2}(\frac{1}{2}-1)\cdots(\frac{1}{2}-n)$, 再将奇数乘积与偶数乘积合并。这说明根号表达式确实生成 Catalan 数, 而不只是满足同一个代数方程。最初几项为 $C(x) = 1 + x + 2x^2 + 5x^3 + 14x^4 + 42x^5 + \cdots$ 。

1.4 Catalan 生成函数的幂：有序森林计数

$C(x)^r$ 的系数统计 r 棵有序满二叉树、内部结点总数为 n 的森林。令 $T = C - 1 = x(1 + T)^2$ ，用 Lagrange 的复合形式

$$[x^n]H(T(x)) = \frac{1}{n}[u^{n-1}]H'(u)\Phi(u)^n, \quad T = x\Phi(T), \quad n \geq 1.$$

取 $H(u) = (1 + u)^r$ 、 $\Phi(u) = (1 + u)^2$ ，得到

$$[x^n]C(x)^r = \frac{r}{n} \binom{2n+r-1}{n-1} = \frac{r}{2n+r} \binom{2n+r}{n}, \quad r \geq 1.$$

$n = 0$ 时系数为 1，右侧闭式同样成立； $r = 0$ 时单独使用 $C^0 = 1$ 。因此遇到 $\sum_{n_1+\dots+n_r=n} \prod_i C_{n_i}$ ，可直接使用这个系数公式。特别地， $[x^n]C^2 = C_{n+1}$ ，也能由 $C^2 = (C - 1)/x$ 立即读出。若每棵树还要额外计一个根或一对括号，则使用 $x^r C(x)^r$ ，提取下标相应减少 r 。

1.5 反射原理与组合数差

长度为 $2n$ 的括号序列对应从 $(0, 0)$ 走到 $(2n, 0)$ 的上、下步序列。总路径数是 $\binom{2n}{n}$ 。不合法路径会第一次越过 $y = 0$ ；把第一次越界之前的前缀关于 x 轴反射，可与从 $(0, 0)$ 走到 $(2n, 2)$ 的路径一一对应，数量为 $\binom{2n}{n-1}$ 。所以

$$C_n = \binom{2n}{n} - \binom{2n}{n-1}.$$

这解释了组合数代码中 Catalan 的差值写法，也说明“前缀和不小于零”是关键限制。

1.6 Catalan 的应用识别

遇到以下任一结构，优先尝试 Catalan：

- 递归为“选一个根，左右两边独立”的有序结构；
- 合法括号、栈序列、前缀余额始终非负且最终回到零；
- 从矩形左下到右上、限制不能越过主对角线的格路；
- $n + 2$ 边形的三角剖分。

例如 n 对括号的合法序列数是 C_n ；固定第一对括号包住 i 对括号时，方案数为 $C_i C_{n-1-i}$ ，这正是卷积递推。`math/17-combinatorics.md` 的 `catalan` 接口可在模质数且预处理到 $2n$ 时直接计算。

2. 错排数：容斥与指数生成函数

设 D_n 是 n 个元素没有固定点的排列数。容斥中选出恰好被强制固定的 k 个位置：

$$D_n = \sum_{k=0}^n (-1)^k \binom{n}{k} (n-k)! = n! \sum_{k=0}^n \frac{(-1)^k}{k!}.$$

因此指数生成函数为

$$\sum_{n \geq 0} D_n \frac{x^n}{n!} = e^{-x} \sum_{m \geq 0} \frac{x^m}{m!} = \frac{e^{-x}}{1-x}.$$

从“最后一个元素与谁交换”也可得到

$$D_n = (n-1)(D_{n-1} + D_{n-2}), \quad D_0 = 1, D_1 = 0.$$

前一种推导适合带多个禁位的容斥，后一种适合线性递推计算。

2.1 按环分解：推广到限制环长的排列

更一般地，若排列只允许环长属于 $L \subseteq \mathbb{Z}_{>0}$ ，一个长度为 j 的标号环有 $(j-1)!$ 种，因此

$$P_L(x) = \exp \left(\sum_{j \in L} \frac{x^j}{j} \right).$$

例如禁止固定点取 $L = \{2, 3, \dots\}$ ，指数为 $-\ln(1-x) - x$ ，得到同一个错排 EGF $e^{-x}/(1-x)$ 。按环长限制排列时，这个推导比重新容斥更直接。

3. 使用时如何选择

- 出现“根加左右两个有序子结构”、括号首次匹配、Dyck 路径时，尝试 $C = 1 + xC^2$ 。
- 出现多段独立 Catalan 结构的总规模计数时，直接提取 $C(x)^r$ 。
- 出现排列禁固定点时，使用错排 EGF；更一般的环长限制使用第 2.1 节。
- 从某处开始满足常系数线性递推时，优先转成有理生成函数，再使用主目录的递推或 Bostan-Mori 模板。

先确认规模、空对象、标号与顺序，再写函数方程。OGF 与 EGF 的乘法含义不同，不能只看表达式相似就互换。标号树与 Prüfer 计数单独见 03-prufer.md。

Prüfer 序列：标号树编码与计数

适用于顶点编号为 $1, \dots, n$ 的无向标号树。 $n \geq 2$ 时，树与长度为 $n-2$ 、每项属于 $[1, n]$ 的 Prüfer 序列一一对应。基础结论见 `math/24-counting-formulas.md`；这里补充构造、证明和可复制代码。

1. 编码：反复删除最小叶子

重复 $n-2$ 次：找到编号最小的叶子 v ，记录它唯一的邻居 u ，删除 v 。最后剩下两个顶点，它们之间的边不记录。

例如边集 $\{(1, 4), (2, 4), (3, 4), (4, 5)\}$ ：依次删除叶子 1, 2, 3，每次记录 4，得到序列 $(4, 4, 4)$ 。

为什么顶点 v 出现 $\deg(v) - 1$ 次？每记录一次 v ，就是删除一个与它相邻的叶子，使它的度数减一。当 v 自己成为叶子并被删除，或作为最后两个顶点之一时，它还保留一条边。因此最初的 $\deg(v)$ 条边中恰有 $\deg(v) - 1$ 条通过记录 v 消去。特别地，原树的叶子恰好是序列中未出现的顶点。

2. 解码：剩余出现次数决定当前叶子

令 c_v 为 v 在整个序列中的出现次数，初始化 $d_v = c_v + 1$ 。从左到右读出序列元素 u ，每次取当前 $d_v = 1$ 的最小顶点 v ，加入边 (v, u) ，删除 v 并将 d_u 减一。最后连接剩余的两个叶子。

当前序列元素 u 在未处理后缀中至少出现一次，所以它此时不是叶子，不会选到 $v = u$ 。每一步选出的最小叶子都与编码时相同，因此两种操作互为逆操作。解码也保证得到一棵树：每条前缀边把一个即将删除的顶点连接到仍保留的顶点，最后再连接剩余两点，不会形成环，且所有点最终连通。

$n = 2$ 对应空序列和唯一边 $(1, 2)$ 。 $n = 1$ 单独视为一棵树；下面的编码接口返回空序列，解码接口用显式参数 n 区分这两种情况。

3. 完整代码：最小堆实现

时间 $O(n \log n)$ ，空间 $O(n)$ 。编码输入必须是一棵合法树，邻接表大小为 $n + 1$ ，下标零不使用；解码要求序列长度为 $\max(0, n - 2)$ 。

```
#include <algorithm>
#include <cassert>
#include <functional>
#include <queue>
#include <utility>
#include <vector>
using namespace std;

vector<int> prufer_encode(const vector<vector<int>>& adj) {
    int n = (int)adj.size() - 1;
    assert(n >= 1);
    if (n <= 2) return {};
    vector<int> degree(n + 1);
    priority_queue<int, vector<int>, greater<int>> leaves;
    for (int v = 1; v <= n; ++v) {
        degree[v] = (int)adj[v].size();
        if (degree[v] == 1) leaves.push(v);
    }
    vector<int> code;
    code.reserve(n - 2);
    for (int step = 0; step < n - 2; ++step) {
        int v = leaves.top();
        leaves.pop();
        int u = 0;
        for (int w : adj[v]) {
            if (degree[w] > 0) { u = w; break; }
        }
        assert(u != 0);
        code.push_back(u);
        degree[v] = 0;
        if (--degree[u] == 1) leaves.push(u);
    }
    return code;
}

vector<pair<int, int>> prufer_decode(int n, const vector<int>& code) {
    assert(n >= 1 && (int)code.size() == max(0, n - 2));
    if (n == 1) return {};
    vector<int> degree(n + 1, 1);
    for (int u : code) {
        assert(1 <= u && u <= n);
        ++degree[u];
    }
    priority_queue<int, vector<int>, greater<int>> leaves;
    for (int v = 1; v <= n; ++v)
        if (degree[v] == 1) leaves.push(v);
    vector<pair<int, int>> edges;
    edges.reserve(n - 1);
    for (int u : code) {
        int v = leaves.top();
```

```

    leaves.pop();
    edges.emplace_back(v, u);
    degree[v] = 0;
    if (--degree[u] == 1) leaves.push(u);
}
int u = leaves.top(); leaves.pop();
int v = leaves.top(); leaves.pop();
edges.emplace_back(u, v);
return edges;
}

```

编码过程中每个顶点至多被删除一次，扫描它原始邻接表的总长度为 $2n - 2$ ；因此邻居搜索总计 $O(n)$ ，主要开销来自堆操作。

4. Cayley 公式与指定度数

每个序列位置有 n 种选择，所以 $n \geq 2$ 时标号无根树总数为

$$n^{n-2}.$$

根固定为某个给定编号时仍为 n^{n-2} ；允许任选一个顶点作为根时为 n^{n-1} 。 $n = 1$ 时这几种计数均单独取 1。

指定度数 $d_1, \dots, d_n$ 时，要求每个 $d_i \geq 1$ 且 $\sum_i d_i = 2n - 2$ 。序列中编号 i 必须出现 $d_i - 1$ 次，因此

$$\# \text{树} = \frac{(n-2)!}{\prod_i (d_i - 1)!}.$$

例如指定顶点 v 的度数为 d ，先选出它出现的 $d - 1$ 个位置，剩下的位置任取其他 $n - 1$ 个编号，故

$$\#\{\deg(v) = d\} = \binom{n-2}{d-1} (n-1)^{n-1-d}, \quad 1 \leq d \leq n-1.$$

5. 指定叶子集合与恰有多少叶子

设指定集合 L 的大小为 ℓ 。如果只要求 L 中的点都是叶子，序列不能出现这些编号，其他编号不受限制，所以计数为

$$(n - \ell)^{n-2}, \quad n \geq 2.$$

这里允许额外叶子。若要求叶子集合恰好是 L ，则余下的 $m = n - \ell$ 个编号 都必须在序列中出现，变成长度 $n - 2$ 的满射计数：

$$m! \left\{ \begin{matrix} n-2 \\ m \end{matrix} \right\} = \sum_{j=0}^m (-1)^j \binom{m}{j} (m-j)^{n-2}.$$

因此恰有 ℓ 个叶子的树共有

$$\boxed{\binom{n}{\ell} (n-\ell)! \left\{ \begin{matrix} n-2 \\ n-\ell \end{matrix} \right\}}.$$

使用 $S(0,0) = 1$ 、空序列计数 $0^0 = 1$ ，上述公式也覆盖 $n = 2$ 、两点均为叶子的情形。 $n = 1$ 的叶子定义另行约定，不套这些式子。

6. 带权 Cayley 与连接已有连通块

给顶点 i 权值 w_i 。一个序列贡献各位置权值之积，由出现次数等于度数减一，对所有序列求和立即得到

$$\boxed{\sum_T \prod_i w_i^{\deg_T(i)-1} = \left(\sum_i w_i \right)^{n-2}}, \quad n \geq 2.$$

两边乘 $\prod_i w_i$ 得到无负指数的形式：

$$\sum_T \prod_i w_i^{\deg_T(i)} = \left(\prod_i w_i \right) \left(\sum_i w_i \right)^{n-2}.$$

常用应用：已有一个森林，含 k 个连通块，大小分别为 $s_1, \dots, s_k$ ，总顶点数为 N 。从完全图中补 $k-1$ 条边使它成为树。先将各块缩成点；块 i, j 之间每条边有 $s_i s_j$ 种端点选择。对于一棵缩点树，选择数是 $\prod_i s_i^{\deg(i)}$ ，故 $k \geq 2$ 时总数为

$$\boxed{N^{k-2} \prod_{i=1}^k s_i}.$$

$k = 1$ 时无需加边，计数为 1。公式要求每对不同块间的所有端点组合均可选，且块内结构已经固定；一般限制边集的生成树计数应使用 Matrix-Tree 定理。